

Assignment 1 - Measures of Central Tendency

Task: Load a small dataset of numbers (e.g., exam scores).

Compute mean, median and mode

~~Hint: Use numpy~~

data set ÷ exam_scores = ([85, 90, 88, 92, ~~85~~ 85, 78, 90, 95, 88, 90])

libraries used = numpy, statistics

numpy.mean(exam_scores) // Use to find the average
median
numpy.~~mode~~(exam_scores) // Use to find the middle
value

statistics.mode(exam_scores) // Use to find the
value which occurs the
most in the data set

Here, we are using statistics library because numpy doesn't have 'mode' function.

Assignment 2 - Measures of Variation

Task: Using the same dataset, compute range, variance, standard deviation

Hint: `var()`, `std()`, `max()` - `min()`

Goal: See how ~~the~~ spread affects interpretation (stable vs volatile data)

Libraries used: `numpy`.

A bit of code:

`data_range = max(data) - min(data)`

`data_variance = np.var(data, ddof=1)` // The ddof

`data_std = np.std(data, ddof=1)`

// `ddof=0` → uses denominator n
→ population variance

~~means~~ stands for
Delta degrees of freedom
which control denominator
when calculating variance and
standard variation.

// `ddof=1` → uses denominator $n-1$
→ sample variance.

// population variation is used when we have all the data
otherwise we use sample ~~formula~~ variance.

Assignment 3 - Summarizing Data with Pandas

Task = Load a CSV dataset. Use `describe()` to generate summary statistics.

Goal = Learn to quickly summarize datasets in EDA phase.

Code +

```
import pandas as pd  
df = pd.read_csv("file.csv")  
print(df.head()) // print first few columns of the dataset  
summary = df.describe()  
print(summary).
```

// `describe()` shows:

count → no. of entries

mean

std

min

25%, 50%, 75% → Outliers

max →

Assignment 4 - Outlier Detection using IQR

~~that~~

Task: Identify outliers in a dataset using Interquartile Range (IQR)

Goal: Connect measures of spread with anomaly detection in Data Science.

Libraries used: Pandas.

Snippets of code:

$Q1 = df["Math"].quantile(0.25)$ // (first quantile, 25%)

$Q3 = df["Math"].quantile(0.75)$ // (^{third} ~~second~~ quantile, 75%)

$IQR = Q3 - Q1$ // (middle 50% values)

$lower_bound = Q1 - 1.5 * IQR$ // (Turkey rule value below that will be an outlier)

$Upper_bound = Q3 + 1.5 * IQR$

// (Same as lower-bound but instead value above is outlier)

$Outliers = df[(df["Math"] < lower_bound | df["Math"] > upper_bound)]$

Very Important:

- ① For data cleaning as outliers can be mistakes
- ② Anomaly detection. They may represent special ~~cases~~ cases.
- ③ ~~in 11 slides~~

Assignment 5 - Data Visualization of Distribution

Task: Plot histogram and boxplot of a dataset. Interpret skewness and outliers.

Goal: Visualize frequency & dispersion

Libraries used: Pandas, Matplotlib.

Code Snippets:

```
df = pd.DataFrame({"Age": age}) // Store data in df
```

```
plt.hist(df["Age"], bins=8, color="skyblue", edgecolor="black")
```

```
plt.title("Histogram of customer ages")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

```
plt.boxplot(df["Age"], vert=False, patch_artist=True)
```

```
plt.title("Boxplot of Customer Ages")
```

```
plt.xlabel("Age")
```

```
plt.show()
```

// We are using matplotlib to visualize the data.

Here we are using histogram to show the shape of the distribution of data (normal, skewed, uniform etc).

// Boxplots is used to summarize data with median, quartiles, spread and outliers.

Assignment 6 - Correlation Matrix (Intermediate)

Task: Load a dataset with multiple numerical columns.
Compute correlation matrix and visualize with heatmap.

Goal: Practice correlation in descriptive stats to see relationships.

Libs used: pandas, seaborn, matplotlib.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Sales": ...,
    "Ads": ...,
    "Profit": ...,
    "Weather": ...
}

df = pd.DataFrame(data)
print(df)

corr_matrix = df.corr()
print(corr_matrix)

plt.figure(figsize=(6,4))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm",
            linewidth=0.5)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

// Strong correlation $\rightarrow$ Ads & Sales,
// Weak $\rightarrow$ Weather vs
Sales/Ads/
Profit

Assignment 7: Group-wise Descriptive Stats

Task: Group a dataset by category. Compute mean, median, variance for each group.

Goals: Learn group-level descriptive stats, often used in business insights and customer segmentation

Libraries used: Pandas

Code:

```
import pandas as pd

data = {
    "Region": [...],
    "Sales": [...],
}

df = pd.DataFrame(data)
print(df)

grouped_stats = df.groupby("Region")["Sales"].agg(
    Mean = "mean",
    Median = "median",
    Variance = "var"
)

print(grouped_stats)
```

// The `groupby()` method divides the DataFrame into groups based on the values in one or more specified column.

// The `agg()` function then applies one or more agg. function to each group.

// Finally the results are combined

```
[2]: import numpy as np

data = [12, 15, 20, 22, 25, 30, 18]

data_range = max(data)-min(data)

data_variance = np.var(data, ddof=1)

data_std = np.std(data, ddof = 1)

print("Range:", data_range)
print("Variance:", data_variance)
print("Standard Deviation:", data_std)

Range: 18
Variance: 36.904761904761905
Standard Deviation: 6.074928962939559
```

```
[2]: import pandas as pd

df = pd.read_csv("student_marks.csv")

print(df.head())

summary = df.describe()

print(summary)
```

	Student	Math	Science	English
0	A	78	82	75
1	B	65	70	68
2	C	89	88	85
3	D	55	58	50
4	E	72	76	70
	Math	Science	English	
count	10.000000	10.000000	10.000000	
mean	73.200000	74.100000	70.600000	
std	11.631375	9.926955	11.983322	
min	55.000000	58.000000	50.000000	
25%	65.750000	67.750000	65.750000	
50%	73.000000	74.500000	71.000000	
75%	80.250000	81.500000	77.250000	
max	90.000000	88.000000	88.000000	


```
[5]: import pandas as pd

# Example dataset
data = {
    "Math": [78, 65, 89, 55, 72, 90, 68, 74, 81, 60, 5, 1000] # notice the "5" is an outlier
}
df = pd.DataFrame(data)

# Step 1: Calculate Q1, Q3, IQR
Q1 = df["Math"].quantile(0.25)
Q3 = df["Math"].quantile(0.75)
IQR = Q3 - Q1

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)

# Step 2: Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

# Step 3: Find outliers
outliers = df[(df["Math"] < lower_bound) | (df["Math"] > upper_bound)]
print("Outliers:")
print(outliers)

Q1: 63.75
Q3: 83.0
IQR: 19.25
Lower Bound: 34.875
Upper Bound: 111.875
Outliers:
   Math
10     5
11  1000
```

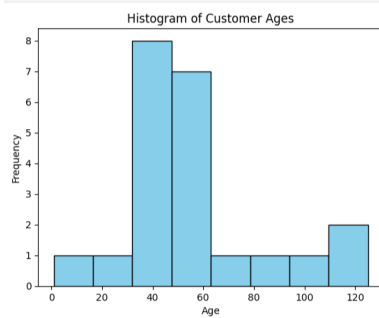
```
[10]: import pandas as pd
import matplotlib.pyplot as plt

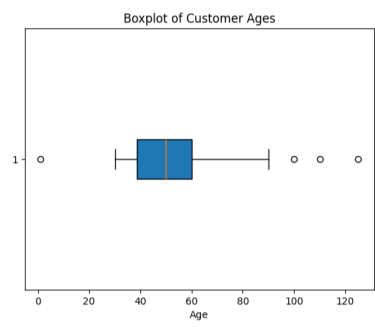
ages = [1,60, 68, 50, 100, 30, 32, 35, 36, 38, 110,
        41, 42, 45, 47, 50, 52, 54, 57, 60, 125, 90]

df = pd.DataFrame({"Age" : ages})

plt.hist(df["Age"], bins=8, color="skyblue", edgecolor="black")
plt.title("Histogram of Customer Ages")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()

plt.boxplot(df["Age"], vert=False, patch_artist=True)
plt.title("Boxplot of Customer Ages")
plt.xlabel("Age")
plt.show()
```






```
[3]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Sales": [200, 220, 250, 270, 300, 320, 340, 360, 400, 420],
    "Ads": [20, 22, 25, 28, 30, 32, 34, 36, 40, 42], # strongly related to Sales
    "Profit": [50, 55, 60, 70, 75, 80, 85, 90, 100, 110], # strongly related to Sales
    "Weather": [30, 25, 28, 32, 29, 27, 31, 26, 30, 28] # almost unrelated to Sales/Ads/Profit
}

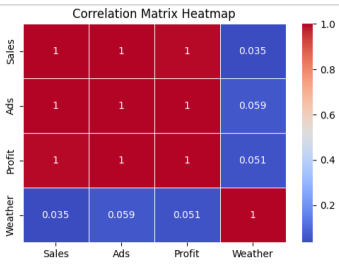
df = pd.DataFrame(data)
print(df)

corr_matrix = df.corr()
print(corr_matrix)

plt.figure(figsize=(6,4))
sns.heatmap(corr_matrix, annot = True, cmap="coolwarm", linewidth=0.5)
plt.title("Correlation Matrix Heatmap")
plt.show()

Sales Ads Profit Weather
0 200 20 50 30
1 220 22 55 25
2 250 25 60 28
3 270 28 70 32
4 300 30 75 29
5 320 32 80 27
6 340 34 85 31
7 360 36 90 26
8 400 40 100 30
9 420 42 110 28

Sales Sales Ads Profit Weather
Sales 1.000000 0.999182 0.999973 0.035197
Ads 0.999182 1.000000 0.997000 0.058611
Profit 0.999973 0.997000 1.000000 0.051381
Weather 0.035197 0.058611 0.051381 1.000000
```



```
[4]: import pandas as pd

data = {
    "Region": ["North", "South", "East", "West", "North", "South", "East", "West", "North", "South"],
    "Sales": [200, 180, 220, 210, 250, 190, 230, 205, 240, 195]
}

df = pd.DataFrame(data)
print(df)

grouped_stats = df.groupby("Region")["Sales"].agg(
    Mean = "mean",
    Median="median",
    Variance="var"
)
print(grouped_stats)
```

	Region	Sales
0	North	200
1	South	180
2	East	220
3	West	210
4	North	250
5	South	190
6	East	230
7	West	205
8	North	240
9	South	195

		Mean	Median	Variance
Region				
East	225.000000	225.0	50.000000	
North	230.000000	240.0	700.000000	
South	188.333333	190.0	58.333333	
West	207.500000	207.5	12.500000	

```
[4]: import pandas as pd

data = {
    "Region": ["North", "South", "East", "West", "North", "South", "East", "West", "North", "South"],
    "Sales": [200, 180, 220, 210, 250, 190, 230, 205, 240, 195]
}

df = pd.DataFrame(data)
print(df)

grouped_stats = df.groupby("Region")["Sales"].agg(
    Mean = "mean",
    Median="median",
    Variance="var"
)
print(grouped_stats)
```

	Region	Sales
0	North	200
1	South	180
2	East	220
3	West	210
4	North	250
5	South	190
6	East	230
7	West	205
8	North	240
9	South	195

		Mean	Median	Variance
Region				
East	225.000000	225.0	50.000000	
North	230.000000	240.0	700.000000	
South	188.333333	190.0	58.333333	
West	207.500000	207.5	12.500000	